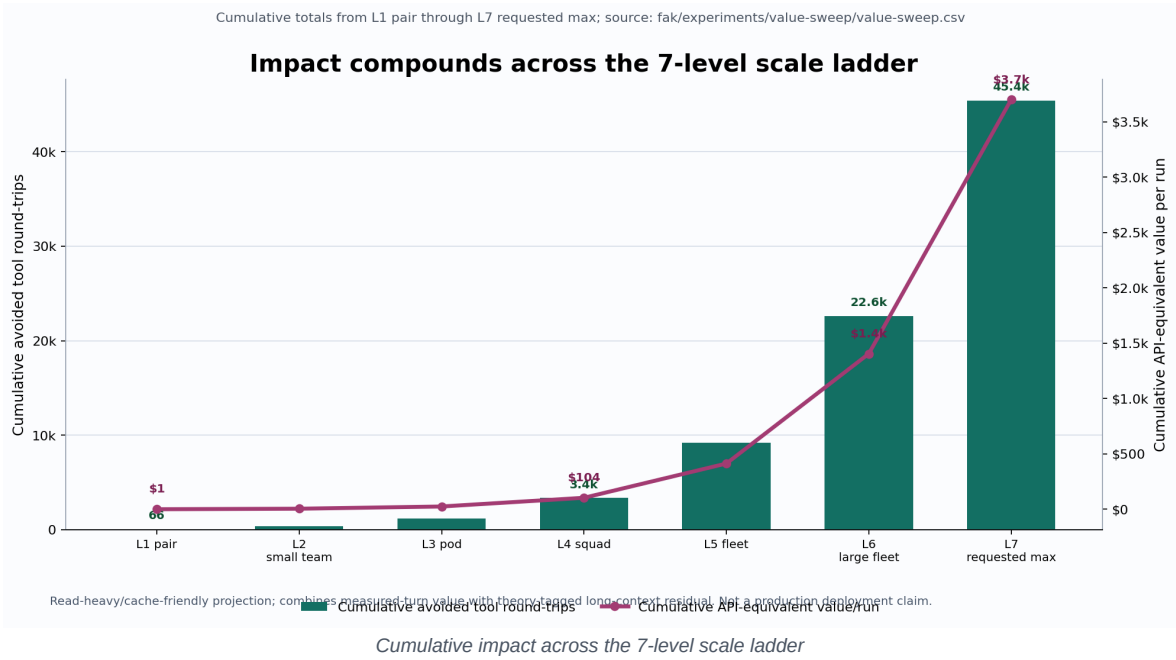


EXPLAIN Executive Summary - Headline Results and Projections

Generated: 2026-06-18

Snapshot used: current worktree on fak-amd-vulkan, described by git as v0.8.0-75-gdcab29b-dirty. This summary uses the active worktree as evidence, not older release tags from other branches. Work-in-progress items are included only when the current files contain measurable or testable evidence.

Impact First: The 7-Level Cumulative Hit



The headline should not be only "91.2% saved" at the top row. The executive hit is that impact compounds as the ladder scales. Excluding the solo baseline, the L1-L7 planning ladder accumulates about **45.4k avoided tool round-trips** and **\$3.7k API-equivalent value per run across the ladder**, ending at the L7 requested-max row of **22,806/25,000 avoided round-trips** and **\$2.29k API-equivalent value for that run**.

This is still a read-heavy, cache-friendly planning projection, not a deployed production claim. The useful point is direction and scale: as the agent fleet gets larger, the value moves from "nice local optimization" to "control-plane and context-work avoidance you can actually budget around."

Top 5 Things That Were Essentially Impossible Before

Before	Now easy / routine	Evidence
Give a cheap or weak model real tools without trusting it to ignore poisoned tool output.	Put a default-deny capability floor and write-time quarantine between the model and tools. A derailable model can still complete the task because the poison never enters context.	fak/LIVE-RESULTS.md: baseline poison reached context 5/5; FAK quarantined 5/5; weak model task completion moved 0/3 -> 3/3.
Reproduce a tool-injection failure and the fix without API keys, live models, or handwavy demos.	Run the deterministic offline A/B and show the baseline executing the destructive op while FAK denies/quarantines/repairs through the same kernel path.	fak agent --offline: 9 -> 7 turns, 40% token reduction, baseline delete_account prevented.
Run many agents through the same read-heavy working set without each one paying the full duplicate tool/result tax.	Share a witnessed world and scoped cache so repeated reads collapse across agents while writes invalidate only the affected resource.	readfleet-50x50.csv: 2,344/2,500 calls deleted; scoped eraser keeps +313 uplift at 1% writes and +235 at 10%.
Decide whether an external API-host bridge is safe enough from prose and screenshots.	Run a one-command bounded proof that generates fixtures, tests requirements, emits a certificate, and names residuals.	api-host-bridge-verify-all.md: 35/35 steps passed; scope-bounded gate green; universal internet behavior explicitly not proven.

Before	Now easy / routine	Evidence
Port the in-kernel model path to a new hardware/runtime target without rewriting the model loop.	Add a backend or ISA lane behind the HAL and prove it with parity tests before making speed claims.	Apple M3 NEON Q8 runs Qwen2.5-1.5B; AMD RX 7600 Vulkan reaches argmax-exact decode and prefill cosine 1.0.

Executive Read

FAK has moved from a research thesis into a runnable agent-tool firewall: a default-deny capability floor, write-time result quarantine, an OpenAI-compatible gateway, in-kernel model work, API-host qualification tooling, and fleet observability are all present in the tree. The strongest headline is not "faster than every serving stack." The strongest headline is: the harness can enforce permissions and contain poisoned tool results from evidence the model did not author, while preserving a measurable path to fleet-scale reuse.

The evidence now supports three external claims:

- A real trust floor exists: policy-denied actions do not dispatch, and poisoned

tool output can be kept out of model context.

- Read-heavy fleet reuse has meaningful measured upside, with scoped invalidation

preserving much of the benefit when writes appear.

- The implementation is becoming operationally legible: CI, benchmark gates,

API-host bridge gates, Grafana/Prometheus surfaces, and release/process guards turn more claims into repeatable artifacts.

The evidence does not support three overclaims:

- It is not a universal proof over every API host or internet deployment.
- It is not production GPU serving parity; the Vulkan AMD backend is correct but

still far from throughput parity.

- It is not a detector-reliability story. Detection remains evadable; the

underwritable claim is capability control plus containment.

Headline Results So Far

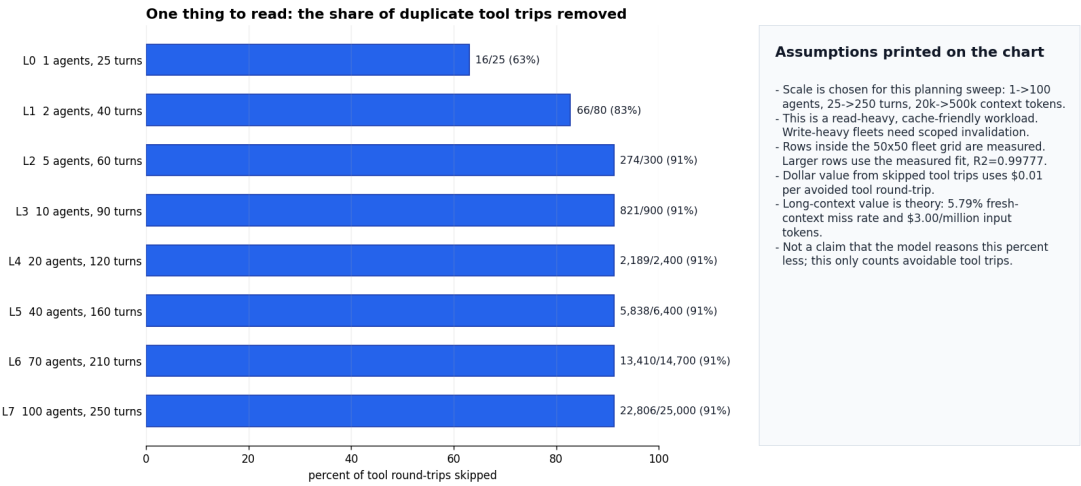
Area	Current evidence	Executive read
Trust floor, live A/B	In fak/LIVE-RESULTS.md, the injection reached baseline context in 5/5 live runs and FAK quarantined it in 5/5. On the weaker Gemini Flash Lite ladder, baseline failed the task 3/3 while FAK completed it 3/3.	Strong shipped demonstration for one attack vector and a small model ladder. This is the clearest product story.
Offline deterministic mechanism	fak agent --offline records 9 -> 7 model turns, 40% token reduction, one grammar repair, one vDSO hit, one quarantine, and baseline delete_account execution prevented by the kernel.	Best no-key repro packet for demos. Use it as a mechanism proof, not as a live-model general law.
Syscall overhead sentinel	fak/report.json records 2,365 ns p50 / 3,451 ns p99 in-process adjudication versus 13,203 ms p50 spawned-hook baseline, gate_primary=pass.	Good regression sentinel that the hot path is resident. Not a production-readiness headline.
Fleet read reuse	readfleet-50x50.csv records 2,344/2,500 calls deleted at the 50x50 read-fleet corner, with +370 cross-agent turns over isolated worlds.	Measured and useful in read-heavy workloads.
Write invalidation	Scoped resource invalidation keeps +313 cross-agent uplift at 1% writes and +235 at 10%; global invalidation turns negative around 1% writes.	The write-heavy objection is real, but the scoped eraser gives a credible mitigation.
Fan-out	fanbench-research.csv at N=1024 records +1,005 sibling-only tool-result saves and 61.7% multi-agent token-tax clawback.	Shows the single-goal fan-out regime between one agent and a full independent fleet.
API-host bridge	api-host-bridge-verify-all.md passes 35/35 generate/test steps; scope-bounded gate is green. Residuals remain universal_any_api_host and paid/keyed live hosts.	Strong bounded proof. It qualifies known fixtures; it does not certify the internet.
Model runtime versus llama.cpp	Zen5 Q8 batched decode now reaches 2,916 tok/s vs llama.cpp ~2,816 tok/s on that axis, but bounded long-context turns x agents are only 0.322x-0.405x of llama.cpp.	Be precise: there is parity on one batched CPU axis, no general speed win. The differentiator is controlled KV/security semantics.

Area	Current evidence	Executive read
Apple Silicon portability	M3 Pro work added arm64 NEON Q8 and pure-Go HF loaders; Qwen2.5-1.5B runs at ~28.9 tok/s decode, within ~2.2x of llama.cpp CPU decode.	Architecture gap closed for a second ISA; still behind tuned kernels.
AMD Vulkan WIP	Current Vulkan result docs and JSON show numerical parity on RX 7600: argmax-exact decode and prefill cosine 1.0. Best WIP throughput is 355.5 ms/tok, 2.8 tok/s, still ~51x behind llama.cpp CPU.	Treat as promising experimental hardware seam. Correctness is real; throughput remains WIP.

Executive Value Projection

Plain English: FAK skips duplicate tool trips in read-heavy fleets

At the max row: 22,806 of 25,000 tool round-trips are avoided (91%). Measured-turn value: \$123/run. Long-context theory: \$2.2k/run API-equivalent.



Hardware sheet: this only prices the long-context theory row

The measured tool-trip savings do not change with GPU choice. Only the self-host cold-prefill math moves.

1. Workload inputs

- Max row: 100 agents doing 250 turns each.
- That is 25,000 total tool round-trips.
- Context size is 500,000 tokens.
- Fresh-context miss rate is 5.79%.
- Cold context avoided: 723.8M tokens.

2. Hardware assumptions

- Profile: 70B_8xH100_default.
- Prefill speed: 20,000 tokens/sec per replica.
- Replica size: 8 GPUs.
- GPU price: \$3.00 per GPU-hour.
- Power: 700 watts per GPU.
- If any of these change, GPU-hours, dollars, and kWh move linearly.

3. What comes out

80.4

GPU-hours avoided

\$241/run

Self-host dollars avoided

56.3 kWh

Power avoided

- Not included: engineering cost, idle capacity, cluster overhead, utilization, or model-quality risk.
- This is not an API bill claim unless the provider exposes a way to avoid this prefill work.

Hardware sensitivity assumptions sheet from checked-in exports.

The current value sweep is useful because it keeps measured turns, measured-supported projection, and long-context theory separate.

At the requested top end - 100 agents, 250 turns per agent, 500,000 context tokens, 25,000 calls - value-sweep.csv projects:

- 22,806 of 25,000 tool round-trips deleted in a read-heavy, cache-friendly regime.
- 792 of those turns are cross-agent-only uplift beyond isolated agents.
- \$123/run from measured tool-turn deletion.

- \$2.17k/run API-equivalent long-context residual under the 5.79% fresh-context

miss-rate model.

- \$241/run self-host GPU-context residual on the default 70B_8xH100_default

hardware model.

- \$2.29k/run total API-equivalent value, or \$364/run total self-host value,

when measured-turn and context-theory rows are combined.

Quote this as a planning projection, not as a deployed production claim. The turn-deletion surface is read-heavy and cache-friendly; the 500k context endpoint is above the measured realistic transcript maximum in this checkout; and a plain API consumer cannot delete provider-internal prefill unless the provider exposes that mechanism.

Projected Items and Likely-Solvable WIP

1. v0.9-style integration release

The checked-in candidate strategy frames the branch as an integration release: API-host bridge qualification, DGX benchmark integrity, model/cache runtime expansion, fleet observability, and trust-floor hardening. This is a credible near-term release shape because the individual gates and fixtures already exist. The release blocker is process discipline: freeze the tree, resolve dirty state, run core gates on a locked snapshot, and keep Vulkan labeled experimental unless the hardware matrix is repeated.

2. AMD/Vulkan backend

This is the cleanest "likely solvable, but not done" item. The hard correctness rung is already green on a real Radeon RX 7600. Multiple WIP fusions moved the decode path in the right direction: stack descriptors, in-place RoPE, residual matmul-add, FFN-tail fusion, and Q/K/V projection fusion culminate in the current 2.8 tok/s measurement. The remaining problem is named: per-dispatch CPU/driver overhead, descriptor update/bind cost, allocation geometry, missing Q8 device GEMM, and no async multi-buffered submission model. That is an engineering program, not a mystery failure.

Good framing: "Vulkan now proves the backend seam on AMD hardware; throughput is an experimental optimization track." Bad framing: "Vulkan is production parity."

3. KV-quarantine bridge into live agent loop

The highest-value next product proof is wiring the synthetic kvmmu bridge into the live fak agent path. The target witness is simple: when a poisoned tool result is fetched, the FAK arm's real attention cache becomes identical to a run that never saw the poisoned span. This would upgrade containment from byte-level page-out to reversible cache-level context pollution.

4. Attack matrix

The live trust-floor result is strong but narrow: one injection vector and a small model ladder. The next credibility step is a reproducible matrix across indirect injection, secret exfiltration, destructive-op lure, tool-result poison, and byte pollution, crossed with the current model ladder. The invariant should stay mechanical: kernel arm never admits the payload; derailable models complete only under the floor.

5. Production gates

The Phase 0 tooling is ready but the clean-node gate is still open. Current local evidence fails the 45x bar at about 40.98x and lacks clean-node provenance. Phase 1 also remains open: no non-reference production backend and no live local-GPU 7-9B rung are proven in the current evidence. These are straightforward gates, not ambiguous narrative gaps.

What To Say Externally Now

Say:

- "FAK is an agent tool firewall: permissions as the floor, filters as additive."
- "The live A/B shows FAK kept poisoned tool output out of context 5/5, and on a weaker model that changed task completion from failure to success."
- "Read-heavy fleet reuse has measured upside; the value sweep is explicitly split between measured turns and theory-tagged long-context residual."
- "AMD Vulkan correctness now runs on real RX 7600 hardware; throughput is an experimental optimization track."

Do not say:

- "Universal API-host security is proven."
- "The detector makes context uninjectable."
- "FAK is broadly faster than llama.cpp/vLLM/SGLang."
- "Vulkan GPU serving is production-ready."
- "The 500k-context projection is a measured production result."

Evidence Index

- README.md - product front door and honest mechanism framing.
- fak/STATUS.md - CI, benchmark, DOS/witness status.
- fak/LIVE-RESULTS.md - live model A/B and offline mechanism proof.
- VISUALS-benchmarking-status-2026-06-18.md - benchmark front door.
- fak/FLEET-VALUE-PROJECTION.md and fak/experiments/value-sweep/value-sweep.csv - projection and raw sweep.
- fak/experiments/fleet/*.csv - fleet sweep and invalidation data.
- fak/experiments/fanout/fanbench-research.csv - fan-out data.
- fak/experiments/api-host-bridge/api-host-bridge-verify-all.md - API-host bounded proof.
- fak/LLAMACPP-HEADTOHEAD-RESULTS.md and fak/M3-LLAMACPP-RESULTS.md - tuned-reference performance posture.
- fak/VULKAN-AMD-RESULTS.md and fak/experiments/parity/vulkan-rx7600-*.json - AMD Vulkan correctness and WIP throughput.
- docs/releases/v0.9.0-candidate-strategy.md - release-shape recommendation for the active branch.